

Food Store — TPI

Entrega parcial · Unidades 1-3

Base de Datos II — UTN · Equipo G · 22/09/2026

Integridad, transacciones y concurrencia (U1) · Optimización de consultas (U2)

Índices, vistas y objetos programables en PL/pgSQL (U3)

Motor: PostgreSQL 16+ · ENUM, TIMESTAMPTZ, IDENTITY, JSONB, CALL, tablas de transición

Documentos que abren la entrega: informe técnico y checklist de objetivos 1-9

Informe técnico — Entrega parcial del TPI «Food Store»

Base de Datos II · UTN · Equipo G · 22/09/2026 · Motor: PostgreSQL 18 (≥16) + PL/pgSQL

Resumen de lo actuado en las Unidades 1–3 con el flujo de cátedra **Kiro (specs)** → **OpenCode (IA)** → **el motor decide (evidencia)**. El checklist de verificación por objetivo está en `docs/checklist_TPI.md` y la bitácora de uso de IA en `duia.md`.

Unidad 1 — Integridad, transacciones y concurrencia

Qué se implementó

- Modelo ER → relacional → normalización a BCNF → DDL definitivo (`schema.sql`), con ENUM, `TIMESTAMPZ`, `IDENTITY`, PK/FK con `ON DELETE RESTRICT`, `UNIQUE` (email, nombre de categoría, par de detalle), `CHECK` y 3 índices justificados.
- Reglas R8–R10 garantizadas por el motor con triggers PL/pgSQL (`sql/restricciones.sql`): transiciones de estado del pedido, stock suficiente con descuento atómico y precio de línea congelado.
- Protocolo de trabajo seguro sobre copia: `protocolo_seguridad.md` (copia → transacción → respaldo).

Cómo se probó

- DDL: ejecución completa de `schema.sql` sobre una copia de trabajo, sin errores.
- Triggers: batería de casos **válidos** (INSERT/UPDATE lícitos dentro de la transacción, COMMIT final) e **inválidos** (cada uno en su transacción que se aborta y ROLLBACK); se verificó el mensaje de excepción de cada regla.
- Concurrencia: 4 escenarios reproducidos con **dos sesiones** `psql` simulando dos terminales (informe `docs/informe_concurrencia.md`).

Resultados

- READ COMMITTED permite **lectura no repetible** (precio 1200→1300) y **fantasma** (COUNT 2→3); REPEATABLE READ los elimina con instantánea fija.
- Un `UPDATE` de la misma fila se **bloqueó 4,51 s** hasta el COMMIT de la otra sesión (mecanismo que evita la pérdida de la actualización); el conflicto se abortó con `40P01`.
- Los casos inválidos fueron rechazados por el motor con el mensaje de la regla correspondiente (R8 transición, R9 sobreventa, R10 precio congelado).

Unidad 2 — Optimización de consultas

Qué se implementó

- Carga masiva de datos para explotar planes no triviales (`sql/carga_masiva.sql` TP3): 50 k productos, 20 k clientes, 200 k pedidos + 512 k líneas.
- 5 consultas analíticas de 3+ tablas (`sql/queries_tp4.sql`) y su versión optimizada por nodo (`sql/queries_tp4_opt.sql`): vista materializada, `work_mem`, reescritura y funciones de ventana.
- Equivalencias verificadas entre versiones distintas y entre funciones de ventana / subconsultas (`sql/equivalencias_tp4.sql`).

Cómo se probó

- `EXPLAIN (ANALYZE, BUFFERS, TIMING)` antes/después por consulta, planes archivados en `docs/planes4/*.txt` ; lecturas críticas de planes en `docs/ejercicio_lectura_critica_join.md` (inversión externa/interna del Nested Loop, Memoize que cachea cliente pero no pedidos, cost ≠ tiempo).
- Equivalencias con `EXCEPT` → 0 filas (parte del enunciado).

Resultados (antes → después, best of)

Consulta	Técnica	Antes	Después	Ganancia
T1 facturación por categoría/mes	MV <code>mv_facturacion_categoria_mes</code>	1.101 ms	0,6 ms	×1.846
T2 top 3 por categoría (window)	<code>work_mem</code> + plan	125 s	29 s	×4,3
T3 top 50 pedidos	reescritura alias	78,1 ms	47,9 ms	×1,63
T5 acumulado mensual por cliente	MV	6.593 ms	1.887 ms	×3,5
Q6 competencia (TP3)	índices propuestos por IA	—	—	×42

Se **descartaron** propuestas de IA que no mejoraban o rompían equivalencia (OPT-T5-A y la variante con `COUNT(DISTINCT)` que ocupaba DENSE_RANK: se detectó por la diferencia 19.743 y se corrigió a `COUNT(*)`).

Unidad 3 — Índices, vistas y objetos programables del motor

Qué se implementó

- Índices con justificación de plan previo (`sql/indices.sql` , `specs/`), incluido un **índice parcial** para reposición de stock.
- 4 vistas (`sql/views.sql`): 3 de reporte + 1 **de seguridad** que oculta datos personales (no expone `telefono`).
- Vista materializada con índice único para `REFRESH CONCURRENTLY` (`sql/materializadas.sql`).
- **Nuevo en esta entrega:** funciones de negocio PL/pgSQL, función que produce **JSONB**, procedimiento invocado con **CALL** y trigger a nivel sentencia con **tabla de transición** (`sql/procedimientos.sql`).

Cómo se probó

- Mediciones `EXPLAIN (ANALYZE)` antes/después (best of 3), prueba de **costo de escritura** (6.000 pedidos + 6.000 líneas) y descarte D1 de sobreindexación verificado en el motor (el plan seguía siendo Seq Scan).
- Vistas: equivalencia contra la consulta manual con `EXCEPT` → 0/0 en las 4 y verificación de que `v_cliente_contacto` no tiene `telefono`.
- MV: equivalencia 0/0, build y refresco concurrente; procedimientos/función/ transición: `sql/prueba_procedimientos_TPI.sql` (CALL, funciones, auditoría).
- La **repetición de corridas** confirmó estabilidad (no single-shot).

Resultados

Ítem	Antes	Después	Diferencia
A1 pedidos del mes ((fecha, estado))	18,8 ms	11,5 ms	×1,6
A2 stock bajo (índice parcial)	10,4 ms	0,34 ms	×30,4
A3 cliente por apellido (opclass)	2,75 ms	1,40 ms	×2,0
Escritura pedido (costo real)	234,5 ms	237,8 ms	+1,4 %
MV facturación ×41.000	905,8 ms	0,022 ms	×41.000
Descartes	D1 (cantidad) y D2 (cliente, estado)	—	documentados en specs/

Vista materializada: `REFRESH CONCURRENTLY` habilitado por índice único, frecuencia justificada (corte mensual, 117 filas → refresco económico).

Uso de herramientas de IA

- **IA usada:** solo **OpenCode** (la herramienta indicada por la cátedra), con el flujo de especificación de **Kiro**. No se utilizó ninguna otra herramienta de IA.
- **Decisión aceptadas** (con verificación en el motor): índices A1–A3, MVs T1/T5, reescritura de T3, procedimiento/funciones y trigger de transición propuestos por la IA y validados con `EXPLAIN`/equivalencia.
- **Decisiones descartadas:** OPT-T5-A (equivalencia), índice D1 `detalle(cantidad)` (el plan no cambiaba), D2 redundante, y la especificación inicial de la parte 3 de TP4 que usaba `COUNT(DISTINCT)` como `DENSE_RANK` (corregida con `COUNT(*)`).
- **Verificación humana:** cada script se aplicó sobre la **copia de trabajo** dentro de `BEGIN/COMMIT` o `BEGIN/ROLLBACK`, sin tocar las bases en uso; los planes y números provienen de corridas reales del motor, no de estimaciones.

Reproducción completa y comandos: `README.md`. Evidencia por objetivo: `docs/checklist_TPI.md`.
Bitácora de sesiones IA: `duia.md`.

Checklist TPI — Food Store · Base de Datos II (UTN)

Verificación de cobertura de los 9 objetivos de la entrega parcial (Unidades 1-3). Para cada objetivo: **evidencia** (archivo) y **cómo verificarlo**. Marcar ✓ al presenciar la corrida.

Motor: PostgreSQL 18 (≥ 16 exigido) con PL/pgSQL. Base de verificación: `food_store_tp5` (creada por copia de `food_store_tp3`).

1. Modelo ER (entidades, atributos, claves, cardinalidad, participación)

- **Evidencia:** `TP1/desarrollo_TP1.pdf` (Parte 1) · `TP1/diagrama_ER.png` · reglas R1–R7 en `schema.sql`.
- **Verificar:** entidades Cliente/Categoría/Producto/Pedido con atributos y claves; cardinalidades 1:N (categoría–producto, cliente–pedido) y N:M (pedido–producto); participación total justificada por R1/R2 y parcial por R6 (mail único).

2. Paso de ER a relacional (1:N y N:M con tabla intermedia)

- **Evidencia:** `TP1/desarrollo_TP1.pdf` (Parte 2) · `schema.sql`.
- **Verificar:** 1:N con FK en el lado N (`producto.categoria_id`, `pedido.cliente_id`); N:M resuelta con tabla asociativa `detalle_pedido` (FK doble + UNIQUE (`pedido_id`, `producto_id`)); clave sustituta `GENERATED ALWAYS AS IDENTITY` justificada.

3. Normalización hasta 3FN/BCNF con justificación de DF

- **Evidencia:** `TP1/desarrollo_TP1.pdf` (Parte 3) · comentarios en `schema.sql` (total derivado = DF transitiva evitada; el par (`nro_pedido`, `producto`) define cantidad y `precio_unitario`).
- **Verificar:** una sola tabla plana $\rightarrow$ 1FN $\rightarrow$ 2FN $\rightarrow$ 3FN $\rightarrow$ BCNF, con la DF (`nro_pedido`, `producto`) $\rightarrow$ (cantidad, `precio_unitario`) y la eliminación de la dependencia transitiva `nro_pedido` $\rightarrow$ cliente, `forma_pago`.

4. DDL completo (tipos, PK/FK, restricciones e índices)

- **Evidencia:** `schema.sql`.
- **Verificar:** ENUM (`forma_pago`, `estado_pedido`), `TIMESTAMPZ`, `IDENTITY`, PK, FK con `ON DELETE RESTRICT` (R7), `UNIQUE` (email, nombre de categoría, par de detalle), `CHECK` (precios/stock no negativos, cantidad positiva), 3 índices justificados (uno parcial: `WHERE activo`).

5. DML y consultas (JOIN, agregación, subconsultas, GROUP BY/HAVING, ventana)

- **Evidencia:**
- JOIN + SUM + GROUP BY: `sql/queries.sql`.
- Subconsultas + HAVING: `sql/queries_tp4.sql` (T4/T5) y `sql/equivalencias_tp4.sql`.
- Funciones de ventana: `ROW_NUMBER() OVER` y `SUM(...) OVER (PARTITION BY ... ORDER BY ...)` en `sql/queries_tp4.sql` y `sql/queries_tp4_opt.sql`.
- **Verificar:** ejecutar contra `food_store_tp5`; las equivalencias ventana↔subconsulta ya fueron validadas con `EXCEPT` 0 filas (`sql/equivalencias_tp4.sql`, sección 3).

6. Vistas, funciones y procedimientos almacenados en PL/pgSQL

- **Evidencia:**
- Vistas: `sql/views.sql` (+ verificación `EXCEPT` 0/0 en `sql/verificacion_views.sql`).
- Funciones de negocio + procedimiento + JSONB + transición: `sql/procedimientos.sql` y `sql/prueba_procedimientos_TPI.sql`.
- Funciones de trigger R8–R10: `sql/restricciones.sql`.
- **Verificar:** `CALL registrar_reposicion(1, 20)`; `SELECT calcular_total_pedido(212006)`; `SELECT resumen_pedido_jsonb(212006)` (devuelve JSONB con líneas y total); auditoría 1 fila por sentencia en `auditoria_carga_lineas`.

7. Reglas de negocio con CHECK, UNIQUE y triggers

- **Evidencia:** CHECK/UNIQUE en `schema.sql`; triggers R8 (transiciones), R9 (stock suficiente), R10 (precio congelado) en `sql/restricciones.sql`; casos válidos e inválidos en `sql/` + informe `docs/DUIA_parte1.md`.
- **Verificar:** insertar una línea sin stock → rechazo `R9`; `UPDATE` de `estado` fuera de flujo → rechazo `R8`; `UPDATE precio_unitario` → rechazo `R10`; mail duplicado → viola `UNIQUE`.

8. Transacciones: atomicidad, COMMIT, ROLLBACK, aislamiento y concurrencia

- **Evidencia:**
- Niveles de aislamiento y anomalías: `docs/informe_concurrencia.md` (READ COMMITTED vs REPEATABLE READ; no repetible, fantasma, espera de lock 4,51 s, `40P01` y `FOR UPDATE`; 2 sesiones psql).
- BEGIN/ROLLBACK como protocolo: `docs/ejercicio_lectura_critica.md` y `protocolo_seguridad.md`.
- Atomicidad con auditoría: sección 6 de `sql/prueba_procedimientos_TPI.sql` (ROLLBACK revierte la fila de auditoría).
- **Verificar:** correr los 4 escenarios del informe con dos ventanas de `psql`; verificar `filas_auditoria_tras_rollback = filas_auditoria_antes`.

9. Borrado lógico (soft delete) y su impacto en consultas e índices

- **Evidencia:** columna `activo BOOLEAN` en `categoria` y `producto` (`schema.sql`); índice parcial `idx_producto_categoria ... WHERE activo = TRUE`; Vista `v_productos_vigentes` que filtra `activo` (`sql/views.sql`); reglas R7 en comentarios del DDL.
- **Verificar:** `UPDATE producto SET activo = FALSE WHERE id = ...`; el producto desaparece de la carta (`v_productos_vigentes`) y no se puede reponer (`CALL registrar_reposicion` lo rechaza), pero el histórico de ventas se conserva (no hay DELETE físico; FKs con RESTRICT).

Nota: elementos específicos de PostgreSQL 16+ exigidos

Característica	Dónde se demuestra
ENUM	<code>schema.sql</code> (<code>forma_pago</code> , <code>estado_pedido</code>)
TIMESTAMPTZ	<code>schema.sql</code> (<code>created_at</code> , <code>pedido.fecha</code>)
Columnas IDENTITY	<code>schema.sql</code> (PK de las 5 tablas)
PL/pgSQL	<code>sql/restricciones.sql</code> , <code>sql/procedimientos.sql</code>
PL/pgSQL con JSONB	función <code>resumen_pedido_jsonb</code>
Procedimiento invocado con CALL	<code>registrar_reposicion</code>
Tablas de transición en triggers	<code>trg_detalle_auditoria_carga</code> (<code>REFERENCING NEW TABLE</code> , nivel STATEMENT)

Los contadores dependen de la copia de trabajo: los ids de pedido de la sección 6 cambian con cada corrida (21xxxx); usar `RETURNING` / `max(id)`.

Evidencia de ejecución — Food Store · TPI

(entrega parcial)

Capturas y resultados reales obtenidos del motor para verificar el funcionamiento. **Corrida:** 22/09/2026 · PostgreSQL 18 · database `food_store_tp5` (copia de trabajo). Toda la evidencia es reproducible con los scripts que se citan; las transacciones de demostración terminan en `ROLLBACK` y no alteran datos.

- Reglas R8–R10 y procedimientos: `sql/evidencia_restricciones_TPI.sql` y `sql/prueba_procedimientos_TPI.sql`.
- Vistas y MV: `sql/verificacion_views.sql`, `sql/verificacion_materializada.sql`.
- Planes EXPLAIN antes/después: `docs/planes4/`, `docs/planes5/`, `docs/planes/`.

1. DDL y carga de datos (objetivos 1–4, 7, 9)

Conteos de la copia masiva sobre la que se midió (212.008 pedidos, 512.029 líneas):

categorias	productos	clientes	pedidos	lineas
3	50006	20003	212008	512029

Catálogo: funciones y procedimientos PL/pgSQL, triggers y vistas existentes:

objeto	tipo	retorno
calcular_total_pedido	FUNCTION	numeric
check_transicion_estado_pedido	FUNCTION	trigger
proteger_precio_unitario	FUNCTION	trigger
registrar_reposicion	PROCEDURE	
resumen_pedido_jsonb	FUNCTION	jsonb
verificar_y_descontar_stock	FUNCTION	trigger
tgrname		
trg_detalle_auditoria_carga	(statement, transición)	
trg_detalle_precio_congelado	(R10)	
trg_detalle_stock	(R9)	
trg_pedido_estado_transicion	(R8)	
table_name		
v_cliente_contacto		
v_reporte_detalle_pedido		
v_reporte_pedidos_cliente		
v_reporte_productos_vigentes		

2. Reglas de negocio R8–R10 (objetivo 7)

Salida de `sql/evidencia_restricciones_TPI.sql` (una transacción, ROLLBACK final; cada caso inválido en su SAVEPOINT):

```
=== R8 (valida): PENDIENTE -> EN_PREPARACION -> ENTREGADO ===
  id | estado
-----+-----
 212011 | EN_PREPARACION
  id | estado
-----+-----
 212011 | ENTREGADO

=== R8 (invalida): ENTREGADO -> CANCELADO (estado terminal) ===
^ esperado: ERROR R8 pedido en estado terminal      [SAVEPOINT/ROLLBACK]

=== R9 (valida): la venta descuenta stock atómicamente ===
  id | pedido_id | producto_id | cantidad | precio_unitario
-----+-----+-----+-----+-----
 512029 | 212011 | 1 | 2 | 1200.00
  id | nombre | stock
-----+-----+-----
 1 | Muzzarella | 68

=== R9 (invalida): stock insuficiente rechazado (producto 2) ===
^ esperado: ERROR R9 stock insuficiente      [SAVEPOINT/ROLLBACK]

=== R10 (invalida): precio_unitario de la línea congelado ===
^ esperado: ERROR R10 precio congelado      [SAVEPOINT/ROLLBACK]

lineas_demo_que_quedan
-----
0      <-- sin huella tras ROLLBACK
```

Mensajes emitidos por el motor ante cada caso inválido:

```
ERROR:  R8: pedido 212011 en estado terminal (ENTREGADO), no puede cambiar a CANCELADO
CONTEXTO: función PL/pgSQL check_transicion_estado_pedido() en la línea 13 en RAISE

ERROR:  R9: stock insuficiente (o producto inactivo) para producto 2, se piden 999999 unidades
CONTEXTO: función PL/pgSQL verificar_y_descontar_stock() en la línea 10 en RAISE

ERROR:  R10: el precio unitario de la línea 1 está congelado (R4)
CONTEXTO: función PL/pgSQL proteger_precio_unitario() en la línea 4 en RAISE
```

3. Transacciones, aislamiento y concurrencia (objetivo 8)

Escenarios reproducidos en `docs/informe_concurrencia.md` con dos sesiones `psql` (los tiempos son de corrida real):

Escenario	Aislamiento	Resultado verificado
Lectura no repetible	READ COMMITTED	precio cambió 1200 → 1300 entre leer y releer
Lectura no repetible / fantasma	REPEATABLE READ	instantánea fija: COUNT estable, sin repetición
Espera por fila bloqueada	READ COMMITTED + <code>FOR UPDATE</code>	UPDATE de B desbloqueó recién 4,51 s después del COMMIT de A

Escenario	Aislamiento	Resultado verificado
Escritura cruzada en conflicto	FOR UPDATE	B abortó con 40P01 (el motor prefiere abortar)

Atomicidad también demostrada en §6: el `ROLLBACK` revierte la fila de auditoría
(`filas_auditoria_tras_rollback = filas_auditoria_antes`).

4. Optimización de consultas (objetivo 5 · Unidad 2)

Plan `EXPLAIN (ANALYZE)` real de T1 (facturación por categoría/mes) antes y después de la vista materializada:

T1 ANTES — Seq Scan 250.070 filas · 540,8 ms (plan completo en `docs/planes4/T1_fact_cat_mes_antes.txt`)

```
Sort (cost=156222.89..156851.87 rows=251591 width=258) (actual time=495.139..529.482 rows=39.00 loops=1)
  Sort Key: (to_char((date_trunc('month'::text, pe.fecha)), 'YYYY-MM'::text)), c.nombre
  Sort Method: quicksort Memory: 27kB
  Buffers: shared hit=8458, temp read=1318 written=1321
-> GroupAggregate (cost=33631.67..71739.27 rows=251591 width=258) (actual time=364.389..529.320 rows=39.00)
  Group Key: c.nombre, (date_trunc('month'::text, pe.fecha))
  Buffers: shared hit=8458, temp read=1318 written=1321
-> Gather Merge (cost=33631.67..62933.59 rows=251591 width=203) (actual time=363.354..457.971 rows=25)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=8458, temp read=1318 written=1321
-> Sort (cost=32631.64..32893.72 rows=104830 width=203) (actual time=316.382..328.052 rows=8335)
  Sort Method: external merge Disk: 3472kB
-> Hash Join (cost=5878.18..13498.53 rows=104830 width=203) (actual time=58.589..231.458)
  Hash Cond: (p.categoria_id = c.id)
-> Hash Join (cost=5865.71..12938.20 rows=104830 width=25) (actual time=57.843..183)
  Hash Cond: (d.producto_id = p.id)
-> Parallel Hash Join (cost=3877.57..10674.87 rows=104830 width=25) (actual t
  Hash Cond: (d.pedido_id = pe.id)
-> Parallel Seq Scan on detalle_pedido d (cost=0.00..6250.39 rows=20833)
-> Parallel Hash (cost=3137.61..3137.61 rows=59197 width=16) (actual ti
-> Parallel Seq Scan on pedido pe (cost=0.00..3137.61 rows=59197)
  Filter: (estado = 'ENTREGADO'::estado_pedido)
  Rows Removed by Filter: 49977

Planning Time: 8.590 ms
Execution Time: 540.832 ms
```

T1 DESPUÉS — vista materializada · 0,29 ms · ×1.846 (plan completo en `docs/planes4/T1_fact_cat_mes_despues.txt`)

```
Sort (cost=2.52..2.62 rows=39 width=54) (actual time=0.253..0.253 rows=39.00 loops=1)
  Sort Key: (to_char(mes, 'YYYY-MM'::text)), categoria
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=4
-> Seq Scan on mv_ventas_categoria_mes (cost=0.00..1.49 rows=39 width=54) (actual time=0.139..0.155 rows=39)
  Buffers: shared hit=1
Planning Time: 2.004 ms
Execution Time: 0.293 ms
```

Resumen de las consultas optimizadas (detalle en `informe_tecnico_TPI.md`):

Consulta	Técnica	Antes	Después	Ganancia
T1 facturación categoría/mes	MV	540,8 ms	0,29 ms	×1.846
T2 top 3 por categoría	work_mem	125 s	29 s	×4,3
T3 top 50 pedidos	reescritura	78,1 ms	47,9 ms	×1,63
T5 acumulado mensual	MV	6.593 ms	1.887 ms	×3,5
Q6 (TP3, competencia)	índice cubriente	14,24 ms	0,34 ms	×42

5. Vistas (objetivo 6)

Salida de `sql/verificacion_views.sql`: equivalencia con la consulta manual (`EXCEPT` → 0 filas) sobre las 4 vistas, y comprobación de que la vista de seguridad **no expone** `telefono`:

```
chequeo | a | b
-----+-----
v1_filas | 50006 | 50006      v1_menos_manual = 0 · v1_manual_menos = 0
v2_filas | 212006 | 212006     v2_menos_manual = 0 · v2_manual_menos = 0
v3_filas | 512023 | 512023     v3_menos_manual = 0 · v3_manual_menos = 0
v4_filas | 20003 | 20003     v4_menos_manual = 0 · v4_manual_menos = 0
      chequeo | columnas_telefono
-----+-----
v4_no_expone_telefono | 0
```

6. Objetos programables (objetivo 6 · nota PostgreSQL 16+)

Salida de `sql/prueba_procedimientos_TPI.sql`:

```
=== 1) Procedimiento CALL: registrar_reposicion (productos 1-3) ===
id | nombre | stock
---+-----+-----
1 | Muzzarella | 90
2 | Napolitana | 88
3 | Jamón y Queso | 85

=== 2) Reposición inválida: cantidad <= 0 y producto descartado ===
ERROR: la cantidad de reposición debe ser > 0 (se recibió 0)
ERROR: producto 999999 inexistente o dado de baja, no se repone

=== 3) Tabla de transición: 1 fila de auditoría por SENTENCIA (3 líneas) ===
n_lineas | monto_total | inserted_at
-----+-----+-----
3 | 8100.00 | 2026-09-22 17:23:53.577821-03

=== 4) Funciones de negocio ===
total_pedido_demo | 8100.00
total_proviene_de_lineas | t (total = SUM de las líneas)
nombre_cliente | Dana · primer_producto_jsonb | Jamón y Queso

=== JSONB completo (pretty) ===
{
  "fecha": "2026-09-22T17:23:53.577821-03:00",
  "total": 8100.00,
  "estado": "PENDIENTE",
  "lineas": [ { "cantidad": 3, "producto": "Jamón y Queso", "subtotal": 4200.00, ... },
```

```

        { ... "Muzzarella" ... }, { ... "Napolitana" ... } ],
"pedido": 212012,
"cliente": { "id": 1, "nombre": "Dana", "apellido": "Da Luz" },
"forma_pago": "EFECTIVO"
}

```

=== 5) La fila de auditoría persiste tras COMMIT ===

```

id | n_lineas | monto_total | inserted_at
-----+-----+-----+-----
 7 |         3 |      8100.00 | 2026-09-22 17:23:53.577821-03

```

=== 6) Atomicidad: ROLLBACK revierte también la auditoría ===

```

filas_auditoria_antes | 3      ·  filas_auditoria_en_tx | 4      ·  filas_auditoria_tras_rollback | 3

```

7. Vista materializada (Unidad 3)

Equivalencia original ↔ MV (0/0) y **REFRESH CONCURRENTLY**:

```

mv_menos_original | 0
original_menos_mv | 0
REFRESH MATERIALIZED VIEW      (concurrente, OK)

```

Plan del MISMO cálculo antes (905,8 ms) y con la MV (0,022 ms — **×41.000**):

C1 ORIGINAL — GroupAggregate sobre 250.070 líneas · 905,8 ms

```

GroupAggregate (cost=32473.41..68733.49 rows=243411 width=230) (actual time=574.436..888.866 rows=117.00 loops=1)
  Group Key: c.nombre, (date_trunc('month'::text, pe.fecha)), pe.forma_pago
  Buffers: shared hit=6279, temp read=1195 written=1198
  -> Gather Merge (cost=32473.41..60822.63 rows=243411 width=199) (actual time=573.769..749.385 rows=250070.00 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    -> Sort (cost=31473.39..31726.94 rows=101421 width=199) (actual time=499.775..529.144 rows=83356.67 loops=1)
      Sort Method: external merge  Disk: 3296kB
      -> Hash Join (cost=5574.52..13331.23 rows=101421 width=199) (actual time=53.820..319.753 rows=83356.67 loops=1)
        Hash Cond: (p.categoria_id = c.id)
        -> Parallel Hash Join (cost=5562.05..12788.72 rows=101421 width=29) (actual time=52.963..318.888 rows=83356.67 loops=1)
          Hash Cond: (d.producto_id = p.id)
          -> Parallel Hash Join (cost=4066.92..11027.34 rows=101421 width=29) (actual time=41.702..317.953 rows=83356.67 loops=1)
            Hash Cond: (d.pedido_id = pe.id)
            -> Parallel Seq Scan on detalle_pedido d (cost=0.00..6400.39 rows=213339 width=29) (actual time=0.007..317.953 rows=83356.67 loops=1)
            -> Parallel Hash (cost=3325.85..3325.85 rows=59286 width=20) (actual time=39.000..317.953 rows=83356.67 loops=1)
              -> Parallel Seq Scan on pedido pe (cost=0.00..3325.85 rows=59286 width=20) (actual time=0.007..317.953 rows=83356.67 loops=1)
                Filter: (estado = 'ENTREGADO'::estado_pedido)
                Rows Removed by Filter: 37318
            -> Parallel Hash (cost=1234.68..1234.68 rows=20836 width=16) (actual time=11.702..317.953 rows=83356.67 loops=1)
              -> Parallel Index Only Scan using ix_producto_cat_cover on producto p (cost=0.00..1234.68 rows=20836 width=16) (actual time=0.007..317.953 rows=83356.67 loops=1)
        -> Parallel Seq Scan on categoria c (cost=0.00..1234.68 rows=20836 width=16) (actual time=0.007..317.953 rows=83356.67 loops=1)
      -> Parallel Seq Scan on pedido pe (cost=0.00..3325.85 rows=59286 width=20) (actual time=0.007..317.953 rows=83356.67 loops=1)
        Filter: (estado = 'ENTREGADO'::estado_pedido)
        Rows Removed by Filter: 37318
    -> Parallel Seq Scan on detalle_pedido d (cost=0.00..6400.39 rows=213339 width=29) (actual time=0.007..317.953 rows=83356.67 loops=1)
      Filter: (estado = 'ENTREGADO'::estado_pedido)
      Rows Removed by Filter: 37318
  Buffers: shared hit=6279, temp read=1195 written=1198
Planning Time: 15.148 ms
Execution Time: 905.820 ms

```

C1 CON MV — Seq Scan sobre 117 filas · 0,022 ms

```

Seq Scan on mv_facturacion_categoria_mes_pago (cost=0.00..2.17 rows=117 width=34) (actual time=0.007..0.012 rows=117 loops=1)
  Buffers: shared hit=1

```

Planning Time: 0.033 ms
Execution Time: 0.022 ms

8. Índices con justificación de plan (Unidad 3 · objetivo 9)

A2 (reposición de stock) usa índice **parcial** `WHERE stock < 50`. Plan real antes (Seq Scan sobre 50.006 productos) y después (Index Scan, 10,4 → 0,34 ms — **×30,4**):

A2 ANTES — Seq Scan · 10,35 ms

```
Limit (cost=1954.27..1954.52 rows=100 width=33) (actual time=10.320..10.328 rows=100.00 loops=1)
  Buffers: shared hit=863
  -> Sort (cost=1954.27..1984.77 rows=12198 width=33) (actual time=10.317..10.320 rows=100.00 loops=1)
    Sort Key: stock, id
    Sort Method: top-N heapsort  Memory: 36kB
    Buffers: shared hit=863
    -> Seq Scan on producto p (cost=0.00..1488.08 rows=12198 width=33) (actual time=0.015..7.584 rows=124)
      Filter: (activo AND (stock < 50))
      Rows Removed by Filter: 37563
      Buffers: shared hit=863
Planning Time: 0.259 ms
Execution Time: 10.351 ms
```

A2 DESPUÉS — Index Scan parcial · 0,34 ms

```
Limit (cost=20.48..54.32 rows=100 width=33) (actual time=0.308..0.319 rows=100.00 loops=1)
  Buffers: shared hit=201
  -> Incremental Sort (cost=20.48..4218.30 rows=12405 width=33) (actual time=0.307..0.311 rows=100.00 loops=1)
    Sort Key: stock, id
    Presorted Key: stock
    Sort Method: quicksort  Average Memory: 29kB  Peak Memory: 29kB
    Buffers: shared hit=201
    -> Index Scan using ix_producto_stock_reposicion on producto p (cost=0.29..3690.31 rows=12405 width=3)
      Filter: activo
      Index Searches: 1
      Buffers: shared hit=201
Planning Time: 0.157 ms
Execution Time: 0.342 ms
```

Índices A1/A3 y costo de escritura (`informe_mediciones.md`):

Ítem	Antes	Después	Diferencia
A1 pedidos del mes (fecha, estado)	18,8 ms	11,5 ms	×1,6
A3 cliente por apellido (opclass)	2,75 ms	1,40 ms	×2,0
Escritura: pedido	234,5 ms	237,8 ms	+1,4 %
Escritura: detalle_pedido	416,8 ms	304,7 ms	ruido (n/s)

Las corridas con planes completos de todas las consultas (T1–T5, A1–A3, D1–C1) están archivadas en `docs/planes4/`, `docs/planes5/` y `docs/planes/`.